# ASSEMBLY AWARE TEXT EDITOR

## 1. ABSTRACT

The Assembly Aware Text Editor is a specialized Integrated Development Environment (IDE) designed for 8086 Assembly language programming. Unlike conventional editors, it provides real-time syntax and semantic analysis while the user types, allowing immediate detection of errors such as invalid instructions, incorrect operands, and addressing mode issues.

The system enhances code readability through syntax highlighting and reduces debugging time by identifying errors early. It also supports optional integration with EMU8086 for assembling and executing programs. This project aims to improve learning efficiency for students and beginners in low-level programming.

## 2. OBJECTIVES

- Design a real-time Assembly code editor
- Implement syntax and semantic error detection
- Provide 8086-specific instruction validation
- Develop a user-friendly interface using Python (Tkinter)
- Integrate with EMU8086 for execution
- Enhance learning experience for beginners

## 3. SCOPE

### Included:

- 8086 Assembly language support
- Real-time syntax highlighting
- Error detection and validation
- Automatic saving mechanism
- EMU8086 integration

Limitations:

- Supports only 8086 architecture
- Limited macro and advanced instruction support
- No optimization or compiler-level analysis

## 4. USERS

### 1. Students (Primary Users)

- Engineering students learning Assembly Language
- Use the editor to write, test, and debug programs
- Benefit from syntax highlighting and error detection
- Helps understand registers, memory addressing, and instruction flow
- Reduces confusion compared to plain text editors

---

### 2. Beginners in Low-Level Programming

- New learners with no prior assembly experience
- Use features like:
    - Instruction reference
    - Error panel
    - Memory validation
- Helps avoid common mistakes (syntax errors, invalid addresses)
- Acts as a guided learning tool

---

### 3. Embedded Systems Learners

- Students working on microcontrollers and hardware-level programming
- Use the editor to simulate and validate assembly code
- Helps understand:
    - Memory mapping
    - Register usage
    - Instruction execution flow

## 5. SOFTWARE INTEGRATION

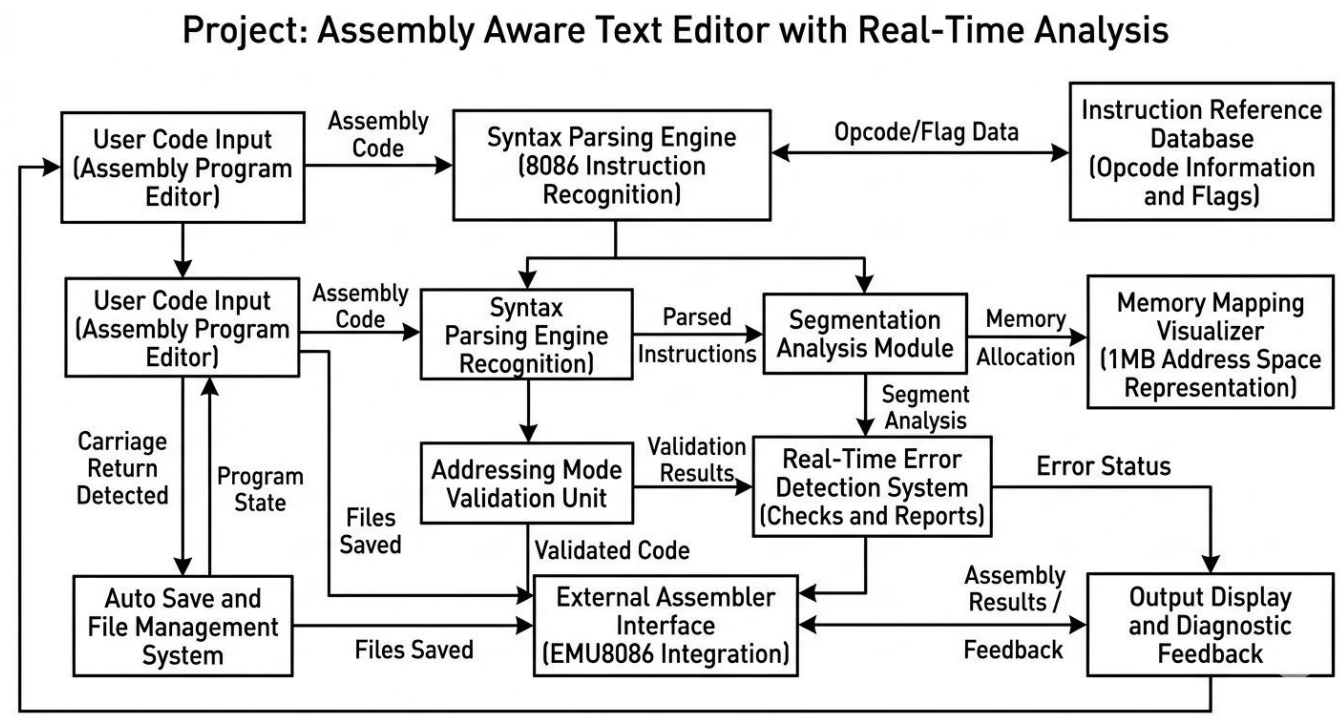
The Virtual CPU Scheduler Simulator integrates the following software components and environments:

| Category | Details |
|---|---|
| **EMU8086 Emulator** | Core runtime environment for executing x86 assembly code. Provides a virtual 8086 CPU, memory model, register panel, and I/O simulation. All assembly modules are assembled and run within this environment. |
| **x86 Assembly Language** | The primary programming language used for all simulation logic — scheduling algorithms, data storage, calculations, I/O, and display. Uses the 8086 instruction set (MOV, CMP, JMP, LOOP, INT, etc.). |
| **BIOS Interrupts (INT 10h)** | Used for low-level screen control — setting cursor position, clearing screen regions, and applying text color attributes for enhanced visual output. |
| **Windows OS** | Host operating system required to run the EMU8086 emulator. The simulator is platform-dependent on Windows due to EMU8086 compatibility. |

## 6. HARDWARE ENVIRONMENT

| Category | Details |
|---|---|
| **Laptop** | Processor : Intel i5-12500H<br>Storage : 512 GigaBytes<br>Ram : 16 GB DDR5<br>Graphics : RTX 3050 |

## 7. BLOCK DIAGRAM WITH DESCRIPTION




## Block Diagram Description

### User UI (Editor Window)

This is where the user writes and interacts with code. The **process** here is capturing keystrokes, edits, and commands, then immediately forwarding them to the controller. It acts as a real-time input/output surface, reflecting processed results like highlights and errors.

### editor_app.py (Main Controller)

This is the brain that coordinates everything. The **process** involves receiving user input, routing it to appropriate modules (logic, visualization, execution), and synchronizing outputs back to the UI. It ensures all components work together without conflict.

### Input Logic

This block handles correctness and structure of code. The **process** is continuous analysis of user input—checking syntax, validating addresses, and enforcing rules before execution or display updates.

- **syntax_highlighter.py**
  The **process** scans code in real-time, identifies tokens (keywords, variables), and applies visual styles instantly. It improves readability without altering logic.
- **linter.py**
  The **process** analyzes code for errors, warnings, and bad practices. It flags issues early, preventing faulty execution.
- **addr_validator.py**
  The **process** validates memory addresses or references, ensuring they are valid and safe before execution. This prevents runtime faults.

### Visualization Components

This block improves how information is presented. The **process** converts raw analysis into visual feedback for the user.

- **line_numbers.py**
  The **process** dynamically tracks and updates line indices as code changes, maintaining alignment with the editor.
- **error_panel.py**
  The **process** collects errors from the linter and displays them in a structured format, allowing quick debugging.

### Background Tasks

This runs silently without interrupting the user. The **process** is periodic execution of supportive tasks.

- **autosave.py**
  The **process** continuously monitors changes and saves the file at intervals, preventing data loss without user action.

### emu_bridge.py (Execution Layer)

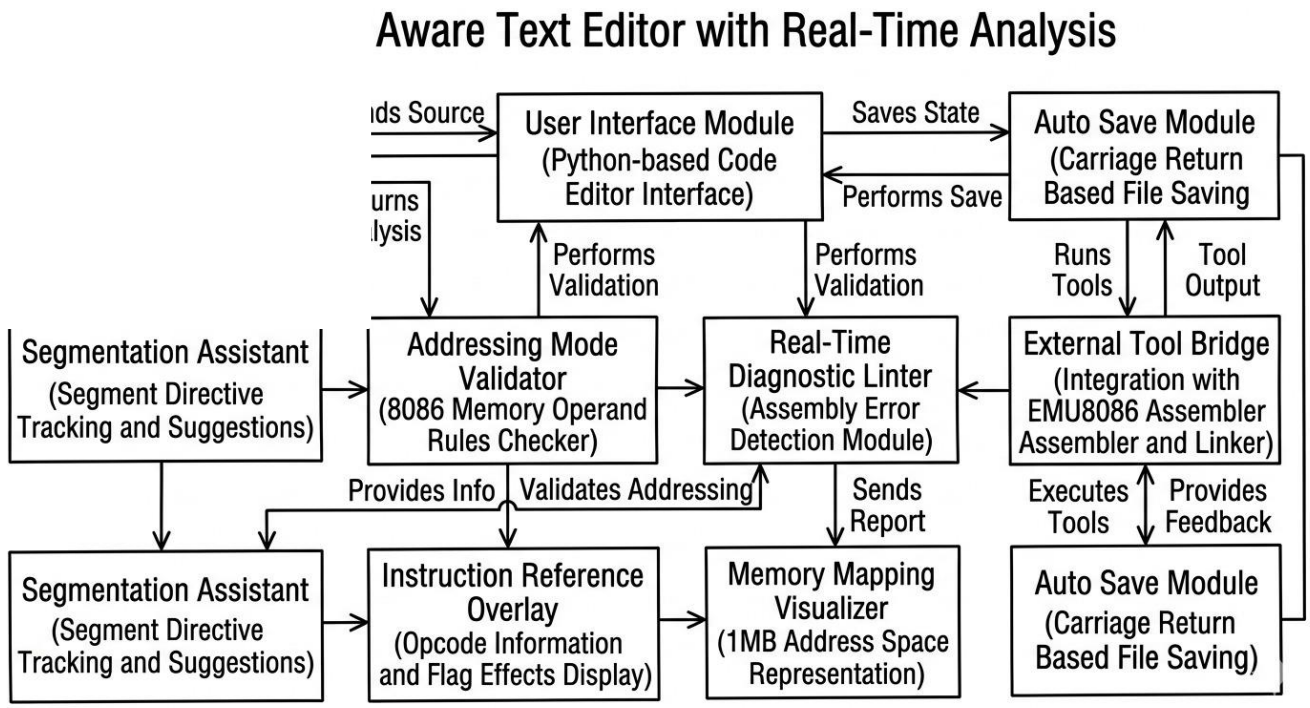
This is the bridge between editor and execution engine. The **process** translates validated code into a format suitable for execution and sends it to the ASM engine, then retrieves results back.

### ASM Engine / Files (.ASM modules)

This is where actual execution happens. The **process** involves assembling and running low-level instructions, then returning output or errors back through the bridge to the UI.

## 8. MODULE DIAGRAM WITH DESCRIPTION

The module diagram below presents a structured view of all ten software modules that constitute the Virtual CPU Scheduler Simulator, along with their individual responsibilities and interactions.




## Module Descriptions

**main.py (Application Entry Point)**
This module initializes the entire system, loads configurations, an
flow. It acts as the starting controller that ensures all components

**core_app.py (Main Version)**
This is the central controller that manages workflow and data flow between all modules. It coordinates communication between Python modules and invokes ASM functions where required.

**autoave.py (Online Guided Save)**
This module continuously monitors user progress and automatically saves results to prevent data loss. It ensures recovery support and maintains session continuity.

**interaction_ref.py (Frame Tester)**
Handles user interactions and acts as a bridge between the user interface and backend logic. It sends inputs to the core system and processes feedback responses.

**emu_bridge.py (EMU8086 Bridge)**
This module connects the application with the EMU8086 emulator to run assembly code. It manages execution, transfers inputs, and retrieves results for further processing.

**error_panel.py (Error Panel)**
Responsible for detecting, displaying, and logging errors generated during execution. It also suggests possible fixes to help users debug efficiently.

**live_numbers.py (Live Numbers)**
Performs real-time calculations and updates dynamic data during execution. It enhances visualization by reflecting changes instantly in the system.

**links.py (ASM Index)**
Maintains a structured list of all assembly modules and their functionalities. It provides quick navigation and access to different ASM components.

**syntax_highlighter.py (Code Highlighter)**
Improves code readability by highlighting assembly syntax and structure. It also supports themes and helps users better understand code formatting.

**Data Handling & Config (JSON, TXT, Directories)**
Manages all configurations, file paths, and stored data required by the system. It ensures smooth loading of test cases and maintains user-specific settings.

**addr_validator.py (Memory Address Check)**
Validates memory addresses used in assembly programs to prevent runtime errors. It checks for alignment and range correctness before execution.

**asm_keywords.py (Mnemonics, Registers, Directives)**
Stores all assembly-related keywords and definitions required for parsing and validation. It supports both syntax checking and code understanding within the system.

## 9. SOURCE CODE

**MODULE 1 - Syntax Highlighter :**

```
.MODEL SMALL

.CODE

ORG 100h

MAIN_LOOP:

   MOV  DX, OFFSET MSG_PROMPT

   CALL PRINT_STR
```

```
    MOV  AH, 0Ah
    MOV  DX, OFFSET INPUT_BUF
    INT  21h
    MOV  DX, OFFSET CRLF
    CALL PRINT_STR
    MOV  SI, OFFSET INPUT_BUF + 1
    XOR  CH, CH
    MOV  CL, [SI]
    INC  SI
    MOV  DI, OFFSET WORD_BUF
    MOV  [WORD_LEN], CL
    JCXZ MAIN_LOOP
TOUPPER_LOOP:
    MOV  AL, [SI]
    CMP  AL, 61h
    JB   STORE_C
    CMP  AL, 7Ah
    JA   STORE_C
    SUB  AL, 20h

STORE_C:
    MOV  [DI], AL
    INC  SI
    INC  DI
    LOOP TOUPPER_LOOP
    MOV  BYTE PTR [DI], 0
    MOV  SI, OFFSET WORD_BUF
    MOV  DI, OFFSET KW_EXIT
```

```
    CALL STR_CMP
    JZ   END_PROGRAM
    CALL CLASSIFY_WORD
    JMP  MAIN_LOOP
END_PROGRAM:
    MOV  DX, OFFSET MSG_BYE
    CALL PRINT_STR
    MOV  AH, 4Ch
    INT  21h
CLASSIFY_WORD PROC
    MOV  SI, OFFSET MNEMONIC_TABLE
    CALL SEARCH_TABLE
    JZ   IS_MNE
    MOV  SI, OFFSET REGISTER_TABLE
    CALL SEARCH_TABLE
    JZ   IS_REG
    MOV  SI, OFFSET DIRECTIVE_TABLE
    CALL SEARCH_TABLE
    JZ   IS_DIR
    MOV  DX, OFFSET MSG_UNKNOWN
    CALL PRINT_STR
    MOV  DX, OFFSET WORD_BUF
    CALL PRINT_STR
    MOV  DX, OFFSET TAIL_UNK
    CALL PRINT_STR
    RET
IS_MNE:
    MOV  DX, OFFSET MSG_MNEMONIC
```

```
    CALL PRINT_STR
    MOV  DX, OFFSET WORD_BUF
    CALL PRINT_STR
    MOV  DX, OFFSET TAIL_MNE
    CALL PRINT_STR
    RET
IS_REG:
    MOV  DX, OFFSET MSG_REGISTER
    CALL PRINT_STR
    MOV  DX, OFFSET WORD_BUF
    CALL PRINT_STR
    MOV  DX, OFFSET TAIL_REG
    CALL PRINT_STR
    RET
IS_DIR:
    MOV  DX, OFFSET MSG_DIRECTIVE
    CALL PRINT_STR
    MOV  DX, OFFSET WORD_BUF
    CALL PRINT_STR
    MOV  DX, OFFSET TAIL_DIR
    CALL PRINT_STR
    RET
CLASSIFY_WORD ENDP
SEARCH_TABLE PROC
NXT:
    MOV  AL, [SI]
    CMP  AL, 0FFh
    JE   NOT_FND
```

```
    XOR  CH, CH
    MOV  CL, AL
    CMP  CL, [WORD_LEN]
    JNE  SKIP
    INC  SI
    MOV  DI, OFFSET WORD_BUF
    PUSH CX
    PUSH SI
CMP_CH:
    MOV  AL, [SI]
    CMP  AL, [DI]
    JNE  MISM
    INC  SI
    INC  DI
    LOOP CMP_CH
    POP  SI
    POP  CX
    XOR  AX, AX
    RET
MISM:
    POP  SI
    POP  CX
    DEC  SI
SKIP:
    XOR  CH, CH
    MOV  CL, [SI]
    INC  CX
    ADD  SI, CX
```

```
    JMP  NXT
NOT_FND:
    OR   AX, 1
    RET
SEARCH_TABLE ENDP
STR_CMP PROC
SL:
    MOV  AL, [SI]
    MOV  BL, [DI]
    CMP  AL, BL
    JNE  SN
    CMP  AL, 0
    JE   SE
    INC  SI
    INC  DI
    JMP  SL
SE:
    XOR  AX, AX
    RET
SN:
    OR   AX, 1
    RET
STR_CMP ENDP
PRINT_STR PROC
    MOV  AH, 09h
    INT  21h
    RET
PRINT_STR ENDP
```

```
INPUT_BUF   DB  20, 0, 21 DUP(0)

WORD_BUF    DB  21 DUP(0)

WORD_LEN    DB  0

KW_EXIT     DB  'EXIT', 0

MSG_PROMPT  DB  0Dh,0Ah,'  Input   : ','$'

CRLF        DB  0Dh,0Ah,'$'

MSG_MNEMONIC  DB '  Result  : [MNEMONIC]  --> ','$'

MSG_REGISTER  DB '  Result  : [REGISTER]  --> ','$'

MSG_DIRECTIVE DB '  Result  : [DIRECTIVE] --> ','$'

MSG_UNKNOWN   DB '  Result  : [UNKNOWN]   --> ','$'


TAIL_MNE    DB ' is a valid 8086 instruction opcode.',0Dh,0Ah,'$'

TAIL_REG    DB ' is a valid 8086 register name.',0Dh,0Ah,'$'

TAIL_DIR    DB ' is a valid assembler directive.',0Dh,0Ah,'$'

TAIL_UNK    DB ' is NOT a recognised 8086 keyword.',0Dh,0Ah,'$'

MSG_BYE     DB  0Dh,0Ah,' Process_Finished',0Dh,0Ah,'$'

MNEMONIC_TABLE:

   DB 3,'MOV', 4,'PUSH',3,'POP', 3,'ADD',3,'SUB',3,'MUL',3,'DIV'

   DB 4,'IDIV',4,'IMUL',3,'AND',2,'OR', 3,'XOR',3,'NOT',3,'NEG'

   DB 3,'CMP',4,'TEST',3,'JMP',2,'JE', 3,'JNE',2,'JZ', 3,'JNZ'

   DB 2,'JA', 3,'JAE',2,'JB', 3,'JBE',2,'JG', 3,'JGE',2,'JL',3,'JLE'

   DB 2,'JS', 3,'JNS',2,'JO', 3,'JNO',4,'JCXZ',4,'CALL',3,'RET'

   DB 4,'RETF',4,'LOOP',5,'LOOPE',5,'LOOPZ',6,'LOOPNE',6,'LOOPNZ'

   DB 3,'NOP',3,'HLT',3,'INT',4,'INTO',4,'IRET',2,'IN',3,'OUT'

   DB 3,'LEA',3,'LDS',3,'LES',4,'XCHG',4,'XLAT',3,'CBW',3,'CWD'

   DB 3,'SHL',3,'SHR',3,'SAL',3,'SAR',3,'ROL',3,'ROR',3,'RCL',3,'RCR'

   DB 4,'MOVS',5,'MOVSB',5,'MOVSW',4,'CMPS',5,'CMPSB',5,'CMPSW'

   DB 4,'SCAS',5,'SCASB',5,'SCASW',4,'LODS',5,'LODSB',5,'LODSW'
```

```
    DB 4,'STOS',5,'STOSB',5,'STOSW',3,'REP',4,'REPE',4,'REPZ'

    DB 5,'REPNE',5,'REPNZ',3,'STC',3,'CLC',3,'CMC',3,'STD',3,'CLD'

    DB 3,'STI',3,'CLI',5,'PUSHF',4,'POPF',3,'AAA',3,'AAD',3,'AAM'

    DB 3,'AAS',3,'DAA',3,'DAS',5,'BOUND'

    DB 0FFh

REGISTER_TABLE:

    DB 2,'AX',2,'BX',2,'CX',2,'DX',2,'AH',2,'AL',2,'BH',2,'BL'

    DB 2,'CH',2,'CL',2,'DH',2,'DL',2,'SP',2,'BP',2,'SI',2,'DI'

    DB 2,'IP',2,'CS',2,'DS',2,'SS',2,'ES'

    DB 0FFh

DIRECTIVE_TABLE:

    DB 7,'SEGMENT',4,'ENDS',4,'PROC',4,'ENDP',2,'DB',2,'DW',2,'DD'

    DB 3,'EQU',6,'ASSUME',3,'END',5,'MODEL',5,'SMALL',5,'LARGE'

    DB 6,'MEDIUM',4,'CODE',4,'DATA',5,'STACK',4,'NEAR',3,'FAR'

    DB 3,'ORG',6,'OFFSET',3,'PTR',4,'BYTE',4,'WORD',5,'DWORD'

    DB 5,'LABEL'

    DB 0FFh

END MAIN_LOOP
```

**Module 2 : Address Validator**

```
.MODEL SMALL

.CODE

ORG 100h

START:

LOOP_TOP:

    MOV  DX, OFFSET MSG_PROMPT

    CALL PRINT_STR

    MOV  AH, 0Ah

    MOV  DX, OFFSET INPUT_BUF
```

```
    INT  21h
    MOV  DX, OFFSET CRLF
    CALL PRINT_STR
    MOV  SI, OFFSET INPUT_BUF+1
    XOR  CH, CH
    MOV  CL, [SI]
    INC  SI
    MOV  DI, OFFSET EXPR_BUF
    JCXZ LOOP_TOP
UP_LOOP:
    MOV  AL, [SI]
    CMP  AL, 'a'
    JB   UC_OK
    CMP  AL, 'z'
    JA   UC_OK
    SUB  AL, 32
UC_OK:
    MOV  [DI], AL
    INC  SI
    INC  DI
    LOOP UP_LOOP                ; Repeat for all characters
    MOV  BYTE PTR [DI], 0       ; Null-terminate EXPR_BUF
    MOV  SI, OFFSET EXPR_BUF
    MOV  DI, OFFSET KW_EXIT
    CALL STR_CMP                ; Compare EXPR_BUF with "EXIT"
    JZ   QUIT

    CALL VALIDATE_ADDR
```

```
    JMP  LOOP_TOP
QUIT:
    MOV  DX, OFFSET MSG_BYE
    CALL PRINT_STR
    MOV  AH, 4Ch
    INT  21h
VALIDATE_ADDR PROC
        MOV  SI, OFFSET EXPR_BUF
        MOV  AL, [SI]
        CMP  AL, '['
        JNE  NOT_MEM              ; Must start with '[' to be a memory reference
        MOV  DI, SI
    FIND_CLOSE:
        MOV  AL, [DI]
        CMP  AL, 0
        JE   NO_CLOSE             ; Reached end of string without finding ']'
        CMP  AL, ']'
        JE   HAS_CLOSE            ; Found closing bracket
        INC  DI
        JMP  FIND_CLOSE
    NOT_MEM:
        MOV  DX, OFFSET MSG_INVALID ; Not a memory expression
        CALL PRINT_STR
        RET
    NO_CLOSE:
        MOV  DX, OFFSET MSG_INVALID ; Missing closing ']'
        CALL PRINT_STR
        RET
```

```
HAS_CLOSE:
    ; -- Extract content between '[' and ']' into INNER_BUF --
    MOV  SI, OFFSET EXPR_BUF
    INC  SI                    ; Skip opening '['
    MOV  DI, OFFSET INNER_BUF
COPY_INNER:
    MOV  AL, [SI]
    CMP  AL, ']'
    JE   INNER_DONE            ; Stop at closing ']'
    MOV  [DI], AL
    INC  SI
    INC  DI
    JMP  COPY_INNER
INNER_DONE:
    MOV  BYTE PTR [DI], 0      ; Null-terminate INNER_BUF
    MOV  SI, OFFSET INNER_BUF   ; SI now points to inner content
    MOV  DI, OFFSET REG_BX
    CALL CHECK_PREFIX          ; Check for "BX"
    JZ   BASE_OK
    MOV  DI, OFFSET REG_BP
    CALL CHECK_PREFIX          ; Check for "BP"
    JZ   BASE_OK
    MOV  DI, OFFSET REG_SI
    CALL CHECK_PREFIX          ; Check for "SI"
    JZ   BASE_OK
    MOV  DI, OFFSET REG_DI
    CALL CHECK_PREFIX          ; Check for "DI"
    JZ   BASE_OK
```

```
        MOV  AL, [SI]
        CMP  AL, '0'
        JB   INVALID_BASE
        CMP  AL, '9'
        JA   INVALID_BASE
        MOV  DX, OFFSET MSG_DIRECT
        CALL PRINT_STR
        RET
    INVALID_BASE:
        MOV  DX, OFFSET MSG_INVALID
        CALL PRINT_STR
        RET
    BASE_OK:
        MOV  DX, OFFSET MSG_DIRECT
        CALL PRINT_STR
        RET
VALIDATE_ADDR ENDP
CHECK_PREFIX PROC
        PUSH SI
        PUSH DI
    CP_LOOP:
        MOV  AL, [DI]
        CMP  AL, 0
        JE   CP_MATCH          ; Reached end of DI string ? full prefix matched
        MOV  BL, [SI]
        CMP  AL, BL
        JNE  CP_MISS           ; Mismatch found
        INC  SI
```

```
        INC  DI
        JMP  CP_LOOP
    CP_MATCH:
        POP  DI
        POP  SI
        XOR  AX, AX              ; AX=0 ? ZF=1 (match)
        RET
    CP_MISS:
        POP  DI
        POP  SI
        OR   AX, 1               ; AX?0 ? ZF=0 (no match)
        RET
CHECK_PREFIX ENDP
STR_CMP PROC
    SL: MOV  AL, [SI]
        MOV  BL, [DI]
        CMP  AL, BL
        JNE  SN
        CMP  AL, 0
        JE   SE
        INC  SI
        INC  DI
        JMP  SL
    SE:
        XOR  AX, AX
        RET
    SN:
        OR   AX, 1
```

```
    RET
STR_CMP ENDP
PRINT_STR PROC
  MOV  AH, 09h
  INT  21h
  RET
PRINT_STR ENDP
INPUT_BUF   DB 30, 0, 31 DUP(0)
EXPR_BUF    DB 32 DUP(0)
INNER_BUF   DB 32 DUP(0)
KW_EXIT     DB 'EXIT', 0
REG_BX      DB 'BX', 0
REG_BP      DB 'BP', 0
REG_SI      DB 'SI', 0
REG_DI      DB 'DI', 0
CRLF        DB 0Dh, 0Ah, '$'
MSG_PROMPT  DB 0Dh,0Ah,'  Enter  : ','$'
MSG_DIRECT  DB '  Result: VALID.',0Dh,0Ah,'$'
MSG_INVALID DB '  Result: INVALID',0Dh,0Ah,'$'
MSG_BYE     DB 0Dh,0Ah,'Completed !!',0Dh,0Ah,'$'
END START
```

**Module  3 : ASM_LINTER**

```
ORG 100h
START:
MAIN:
  MOV  DX, OFFSET MSG_PROMPT
  CALL PRINT_STR
```

```
    MOV  AH, 0Ah
    MOV  DX, OFFSET INPUT_BUF
    INT  21h
    MOV  DX, OFFSET CRLF
    CALL PRINT_STR
    MOV  SI, OFFSET INPUT_BUF+1
    XOR  CH, CH
    MOV  CL, [SI]
    INC  SI
    MOV  DI, OFFSET LINE_BUF
    JCXZ MAIN
UC_LOOP:
    MOV  AL, [SI]
    CMP  AL, 61h
    JB   UC_OK2
    CMP  AL, 7Ah
    JA   UC_OK2
    SUB  AL, 20h
UC_OK2:
    MOV  [DI], AL
    INC  SI
    INC  DI
    LOOP UC_LOOP
    MOV  BYTE PTR [DI], 0
    MOV  SI, OFFSET LINE_BUF
    MOV  DI, OFFSET KW_EXIT
    CALL STR_CMP
    JZ   QUIT
```

```
    CALL LINT_LINE
    JMP  MAIN
QUIT:
    MOV  DX, OFFSET MSG_BYE
    CALL PRINT_STR
    MOV  AH, 4Ch
    INT  21h
LINT_LINE PROC
    MOV  SI, OFFSET LINE_BUF
    MOV  DI, OFFSET PAT_MOV_CS
    CALL CHECK_PREFIX
    JZ   ERR_MOV_CS
    MOV  SI, OFFSET LINE_BUF
    MOV  DI, OFFSET PAT_MOV_AX_AL
    CALL CHECK_PREFIX
    JZ   ERR_SIZE_MISM
    MOV  SI, OFFSET LINE_BUF
    CALL COUNT_BRACKETS
    CMP  AL, 2
    JGE  ERR_DOUBLE_MEM
    MOV  SI, OFFSET LINE_BUF
    MOV  DI, OFFSET PAT_PUSH_ONLY
    CALL STR_CMP
    JZ   ERR_PUSH_NO_OP
    MOV  DX, OFFSET MSG_OK
    CALL PRINT_STR
    RET
ERR_MOV_CS:
```

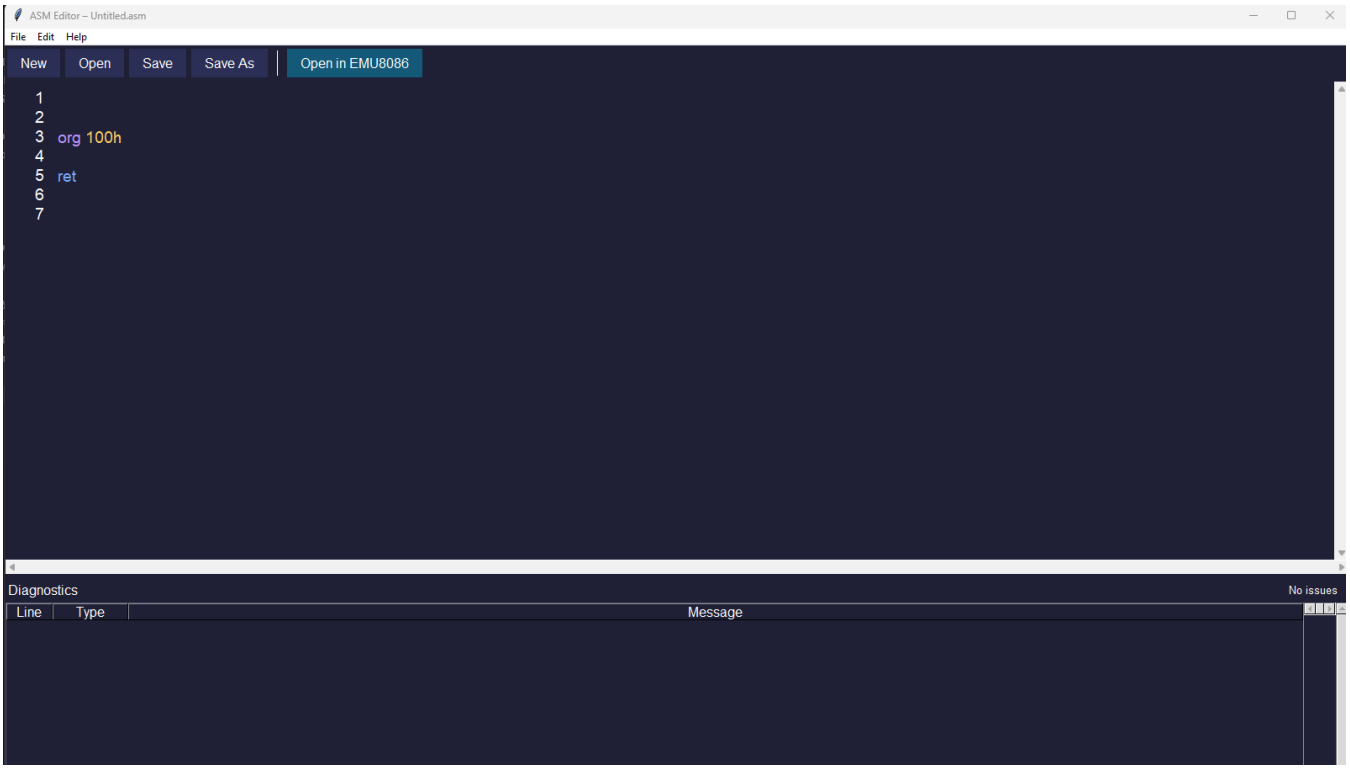
```
    MOV  DX, OFFSET MSG_ERR_CS
    CALL PRINT_STR
    RET
ERR_SIZE_MISM:
    MOV  DX, OFFSET MSG_ERR_SZ
    CALL PRINT_STR
    RET
ERR_DOUBLE_MEM:
    MOV  DX, OFFSET MSG_ERR_MEM
    CALL PRINT_STR
    RET
ERR_PUSH_NO_OP:
    MOV  DX, OFFSET MSG_ERR_PUSH
    CALL PRINT_STR
    RET
LINT_LINE ENDP
COUNT_BRACKETS PROC
    XOR  AL, AL
CB_L:
    MOV  BL, [SI]
    CMP  BL, 0
    JE   CB_DONE
    CMP  BL, '['
    JNE  CB_NEXT
    INC  AL
CB_NEXT:
    INC  SI
    JMP  CB_L
```

```
CB_DONE:
   RET
COUNT_BRACKETS ENDP
CHECK_PREFIX PROC
   PUSH SI
   PUSH DI
CPX:
   MOV  AL, [DI]
   CMP  AL, 0
   JE   CPX_OK
   MOV  BL, [SI]
   CMP  AL, BL
   JNE  CPX_NO
   INC  SI
   INC  DI
   JMP  CPX
CPX_OK:
   POP DI
   POP SI
   XOR AX,AX
   RET
CPX_NO:
   POP DI
   POP SI
   OR  AX,1
   RET
CHECK_PREFIX ENDP
STR_CMP PROC
```

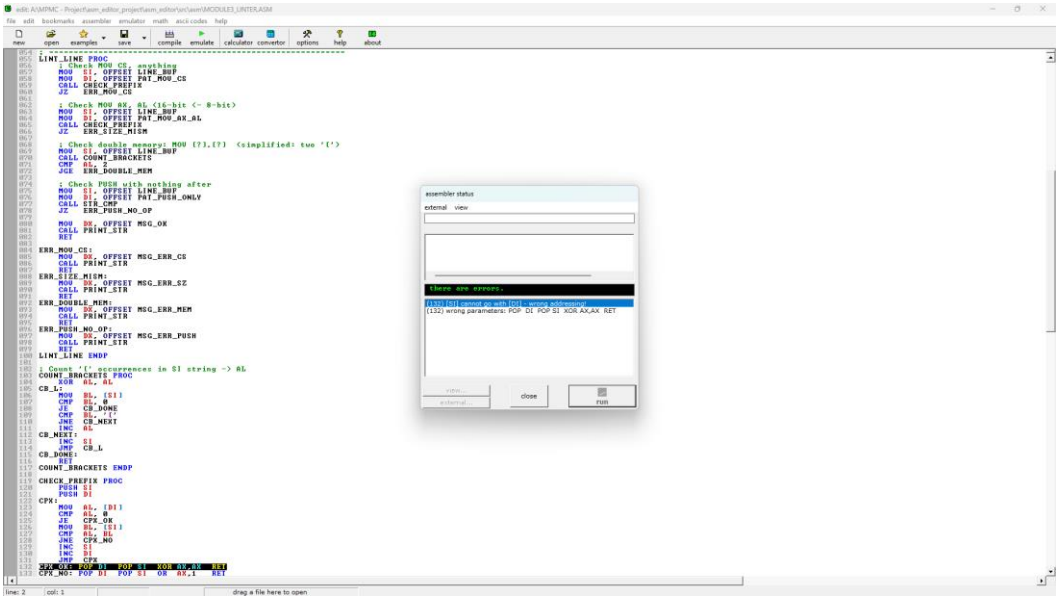
```
SC:
    MOV AL,[SI]
    MOV BL,[DI]
    CMP AL,BL
    JNE SCN
    CMP AL,0
    JE  SCY
    INC SI
    INC DI
    JMP SC
SCY:
    XOR AX,AX
    RET
SCN:
    OR  AX,1
    RET
STR_CMP ENDP
PRINT_STR PROC
    MOV AH,09h
    INT 21h
    RET
PRINT_STR ENDP
INPUT_BUF     DB 40, 0, 41 DUP(0)
LINE_BUF      DB 42 DUP(0)
KW_EXIT       DB 'EXIT', 0
PAT_MOV_CS    DB 'MOV CS', 0
PAT_MOV_AX_AL DB 'MOV AX, AL', 0
PAT_PUSH_ONLY DB 'PUSH', 0
```

```
CRLF          DB 0Dh,0Ah,'$'

MSG_PROMPT    DB 0Dh,0Ah,'  Instruction: ','$'

MSG_OK        DB '  Result: OK - no error .',0Dh,0Ah,'$'

MSG_ERR_CS    DB '  ERROR : CS cannot be a destination register.',0Dh,0Ah,'$'

MSG_ERR_SZ    DB '  ERROR : Size mismatch - 16-bit dst, 8-bit src.',0Dh,0Ah,'$'

MSG_ERR_MEM   DB '  ERROR : MOV cannot have two memory operands.',0Dh,0Ah,'$'

MSG_ERR_PUSH  DB '  ERROR : PUSH requires an operand.',0Dh,0Ah,'$'

MSG_BYE       DB 0Dh,0Ah,' Finished!',0Dh,0Ah,'$'

END START
```

## 10. OUTPUT

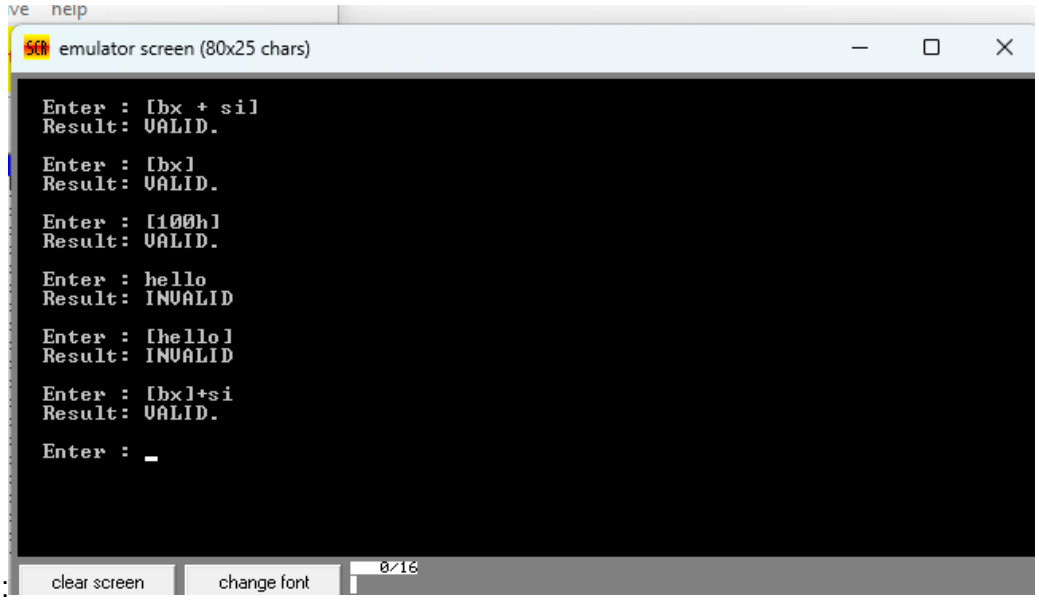


# 11. BUGS AND RECTIFICATION



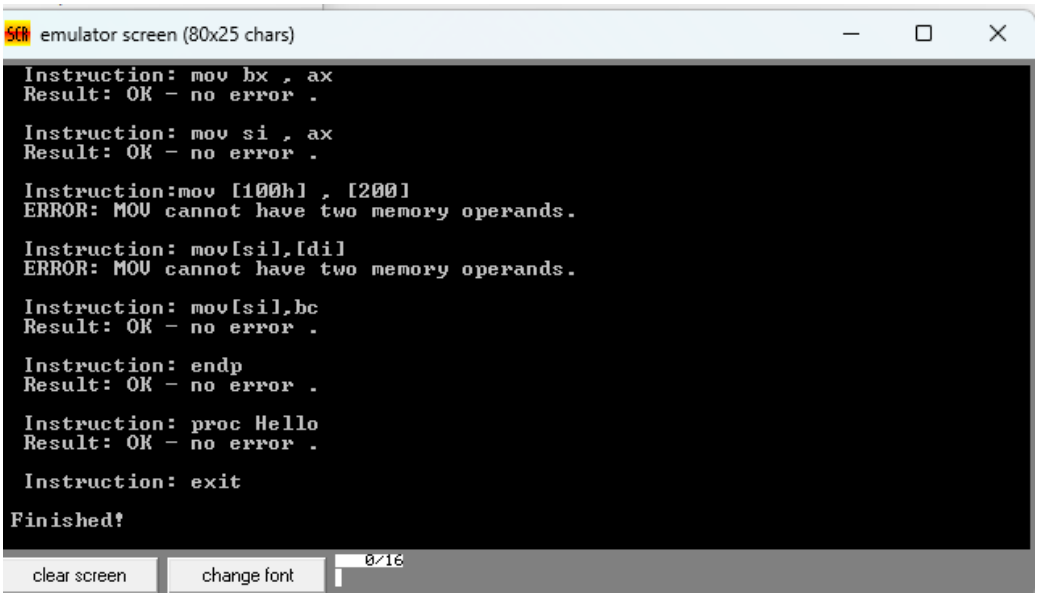

**Multi Line Instruction** :

Multiple instructions were entered within one line, leading to wrong parsing, thus causing an address mismatch problem for the MOV command. This happened due to the requirement that only one instruction was supposed to be on each line .




**Addressing mode module :**

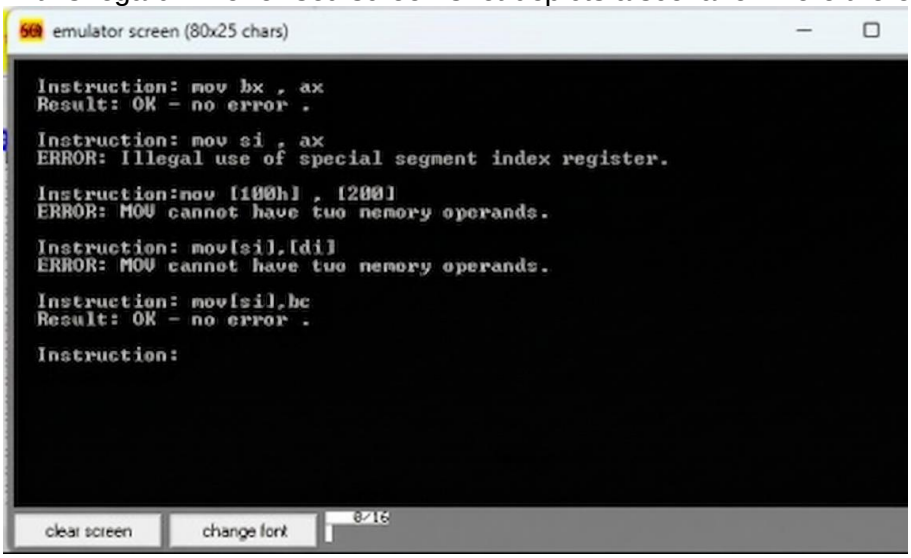
As shown in the screenshot, the validation module for addressing mode works properly when valid data is entered and no errors are produced by the program. This was initially proven to check whether the module functions accordingly.




**Invalid usage of the address :**

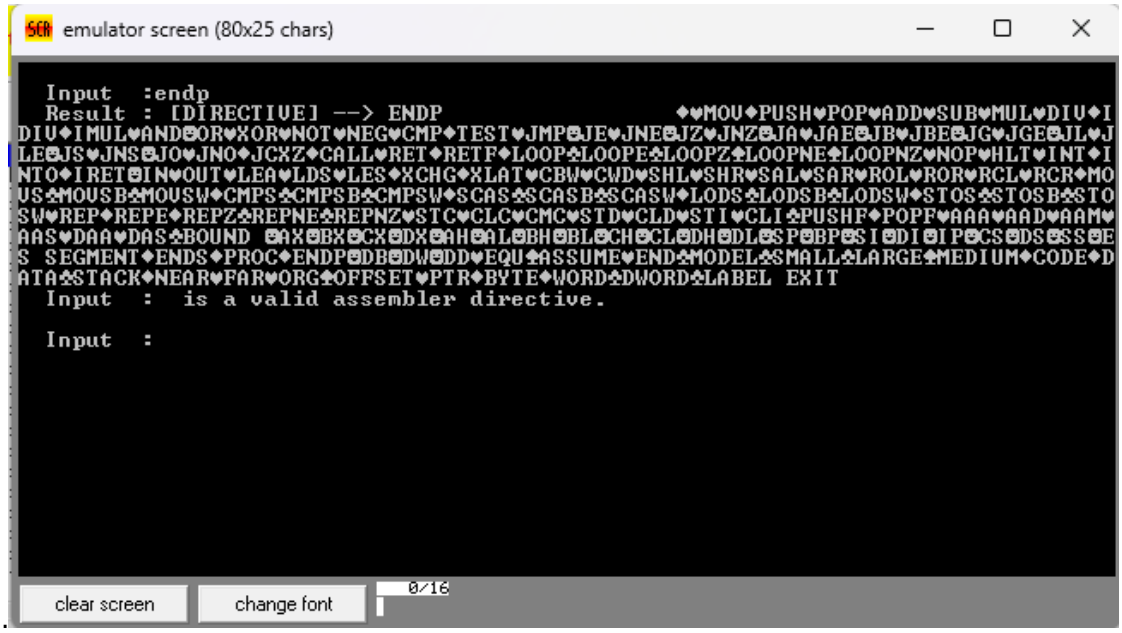
The screen-shot provided is of an instance where there was an improper use of registers (SI and AX) within an addressing environment. The issue here is that the system could not identify the incorrectness

in this regard. The revised screen-shot depicts a scenario where there is proper validation.



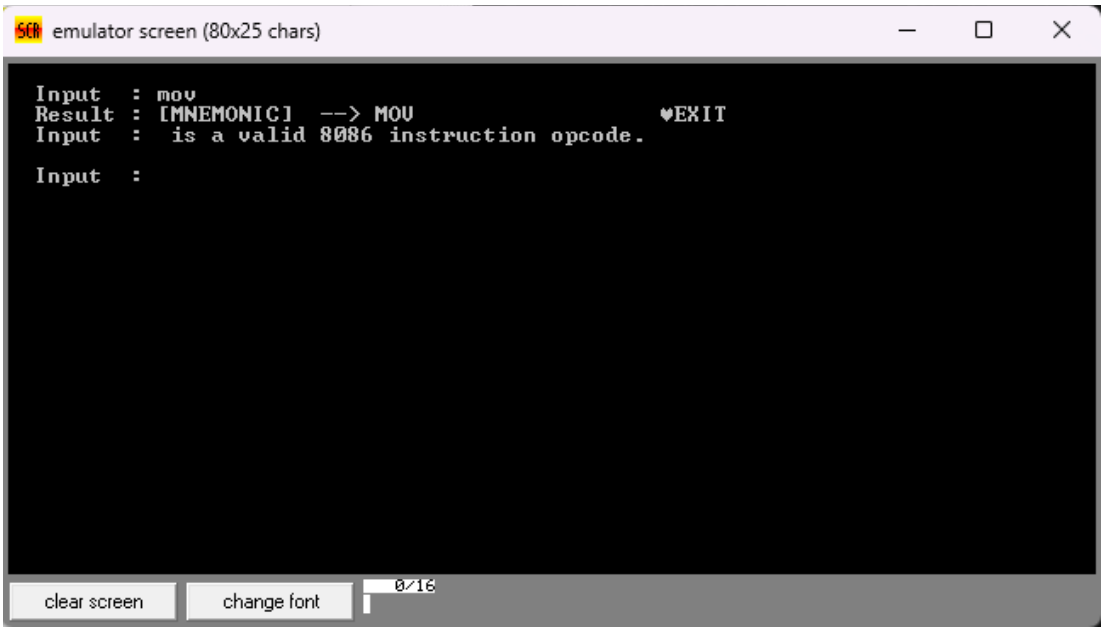

**Invalid usage of the address**

The snapshot represents the same erroneous data entry scenario after applying certain conditional checks. All such errors which earlier went unnoticed are now caught and reported through the program. The corrected snapshot is evidence that the condition has been applied effectively




**Printing the whole Table :**

From the image above, it is clear that when a mnemonic or directive was typed, the program would wrongly display the entire list rather than just the corresponding item. This was because there was no proper way out of the loop. However, in the corrected screenshot below, only the matched instruction will be displayed.

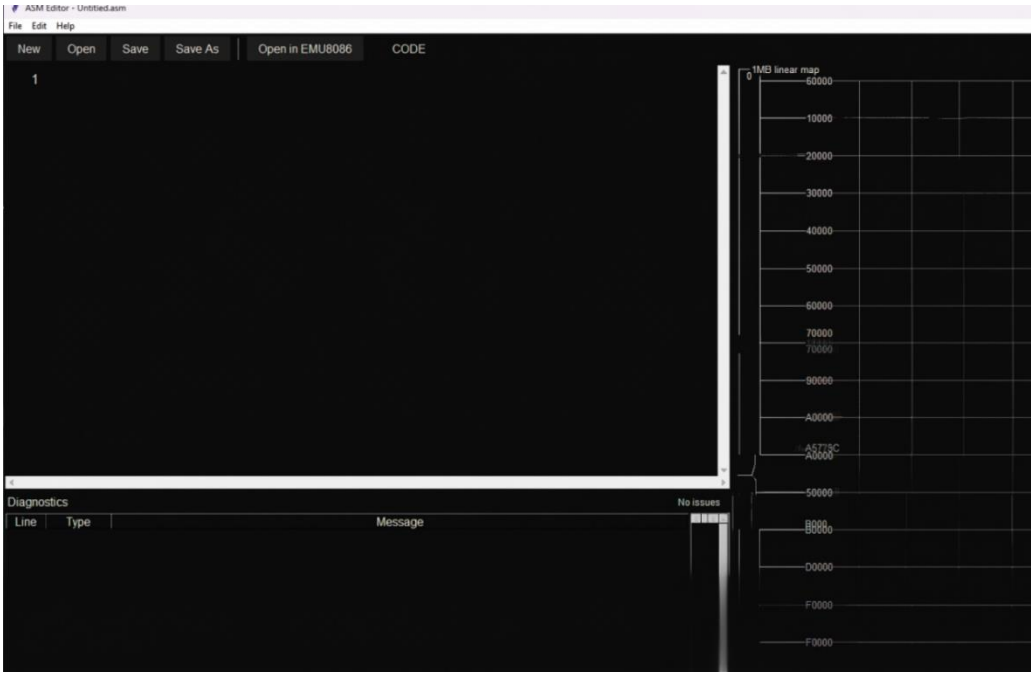



**Printing the Mnemonics :**

The screenshot shows the revised behavior that prints out only the discovered mnemonic rather than listing several outputs. This was done through the inclusion of a jump instruction that stops the iteration process on the table once there is a hit

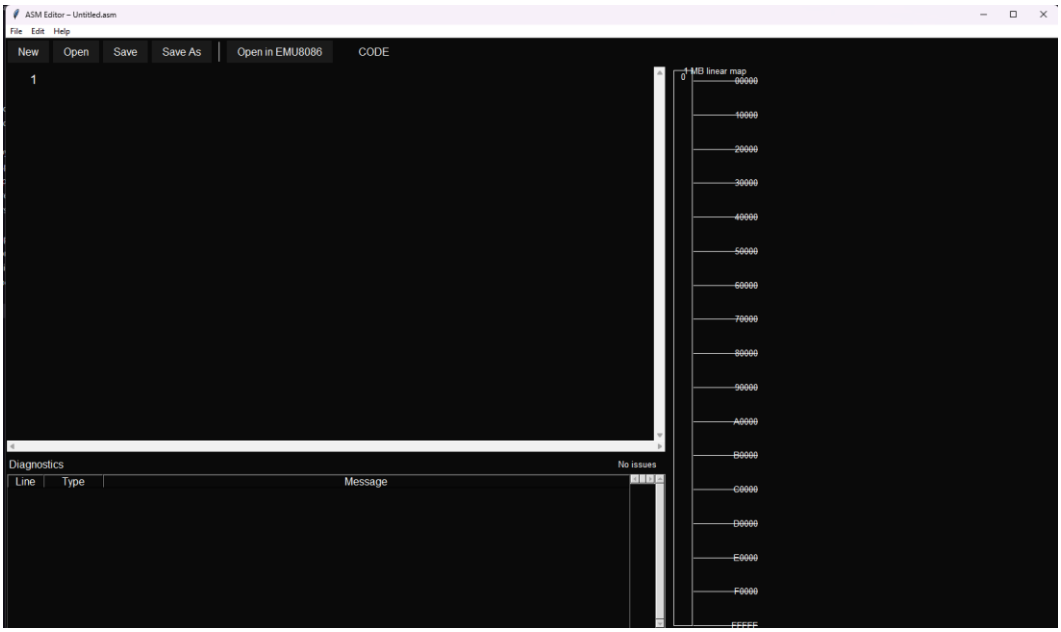
**Invalid parameter in Tkinter:**

The screenshot below shows the error that occurred because of the use of a Tkinter function without having established proper linking of the module with an improper keyword parameter.



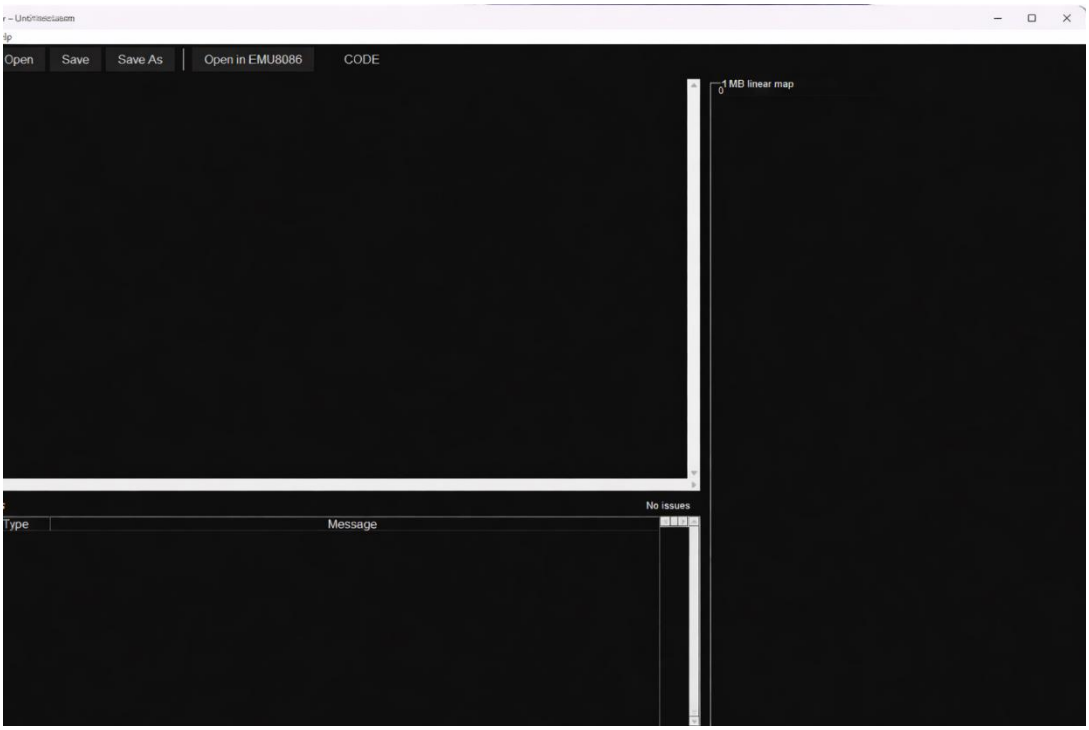

**Vertical Alignment Displacement Error :**

This glitch causes memory addresses on the 1 MB linear map to move and overlap, creating duplicated hex values such as 70000. There is no consistent vertical spacing because sometimes the rows will be cramped together while at other times the text will look ghosted or have corrupt characters such as A9?20C. This appears to be an internal user interface drawing problem because of the way that the gridlines are messed up.




**Skeleton view of the Final output :**

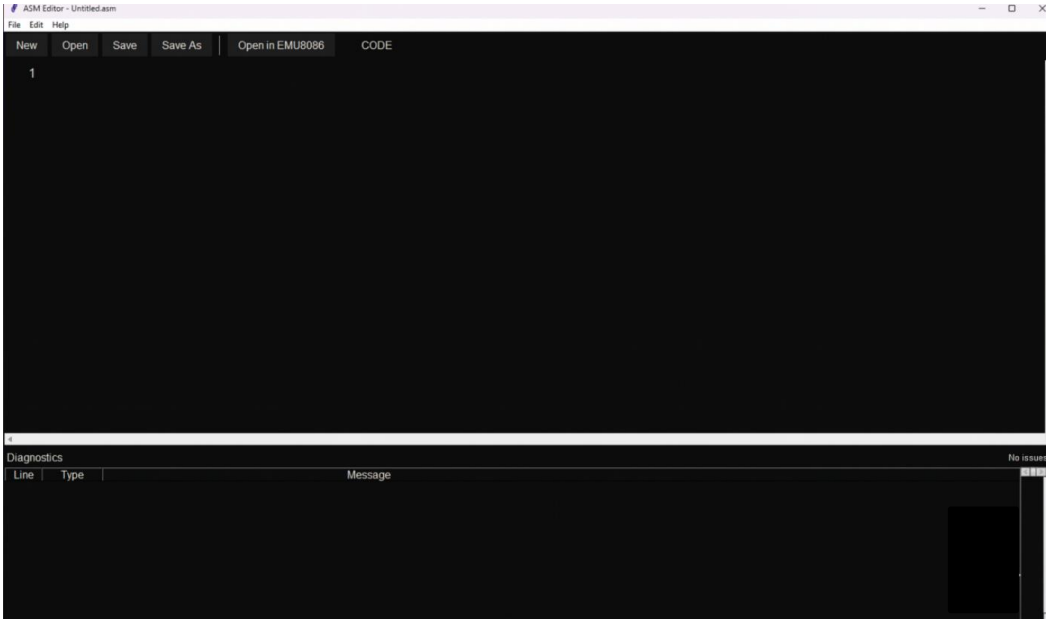
The screenshot demonstrates the first GUI design implemented by using Tkinter that did not have any styling applied to it and had some memory problems. The memory problem was causing instability in the design and functionality of the GUI.




**Memory Map Panel Not Rendering (UI Component Failure)**

However, when trying to delete the memory map panel, the right side does not display correctly.

The space where the panel should be is blank with some leftovers of UI elements, hinting at a problem with rendering.Other than that, the program operates fine, showing that the problem lies solely with the memory map panel.



**ASM Editor Interface Cleanup**

The picture shows the x86 Assembly Editor with its user interface simplified, highlighting only the development space. This has been achieved by eliminating the memory map, which makes the programming interface more concentrated on coding and debugging.

## 12.CONCLUSION

The Assembly Aware Text Editor with Real-Time Analysis meets its objective of creating an intuitive and efficient IDE for the 8086 Assembly language code. The implementation of real-time analysis of syntax and semantics errors, dynamic syntax highlighting, auto-saved files, and context-based verification of commands and addresses makes the system very helpful in solving typical problems experienced by students and novices. The inclusion of a connection to the EMU8086 assembler greatly adds to the applicability of the system through its capability to assemble and run code in one click.

This project has helped in bridging the gap between theory and practice by introducing an easy way of learning programming on low-level languages. Although some issues like multi-line parsing and setting up Tkinter were faced, the end result is a stable system that provides valuable knowledge about Assembly programming. Possible improvements could be in the introduction of other architectures and debugger capabilities. This research will go a long way in computer science education by offering learners an easy-to-use environment for Assembly programming.